

# Code Assessment of the Aave Stable Vaults

July 03, 2026

Produced for



by



# Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>             | <b>3</b>  |
| <b>2</b> | <b>Assessment Overview</b>           | <b>5</b>  |
| <b>3</b> | <b>Limitations and use of report</b> | <b>17</b> |
| <b>4</b> | <b>Terminology</b>                   | <b>18</b> |
| <b>5</b> | <b>Open Findings</b>                 | <b>19</b> |
| <b>6</b> | <b>Resolved Findings</b>             | <b>22</b> |
| <b>7</b> | <b>Informational</b>                 | <b>36</b> |
| <b>8</b> | <b>Notes</b>                         | <b>39</b> |

# 1 Executive Summary

Dear all,

Thank you for trusting us to help Aave Labs with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Aave Stable Vaults according to [Scope](#) to support you in forming an opinion on their security risks.

Aave Labs implements a cross-chain semi-fixed-rate stablecoin vault where users deposit stablecoins on the Accounting Chain (Arbitrum) and earn predetermined per-second interest rates, while protocol managers deploy those assets into ERC-4626 yield strategies across Accounting and Earning Chains. Withdrawals are two-phase: users first request IOUs representing their claim, then execute the withdrawal by burning IOUs for underlying assets, subject to configurable fees.

The most critical subjects covered in our audit are functional correctness, access control, asset solvency, and external integrations.

After the intermediate report, all previously raised issues have been addressed. Access control is high, with tiered execution delays and a guardian hierarchy for emergency revocation. Asset solvency mechanisms are satisfactory, enforcing global solvency invariants during withdrawal requests. Functional correctness is satisfactory. External integrations are satisfactory. The system relies on Chainlink CCIP and oracle infrastructure.

The general subjects covered are documentation and code quality. Both documentation and code quality are satisfactory.

Note that ChainSecurity conducted a separate audit of an updated version of Aave Stable Vaults.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

## 1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

|                                    |    |
|------------------------------------|----|
| <b>Critical</b> -Severity Findings | 0  |
| <b>High</b> -Severity Findings     | 0  |
| <b>Medium</b> -Severity Findings   | 2  |
| • <b>Code Corrected</b>            | 2  |
| <b>Low</b> -Severity Findings      | 12 |
| • <b>Code Corrected</b>            | 9  |
| • <b>Specification Changed</b>     | 1  |
| • <b>Risk Accepted</b>             | 1  |
| • <b>Acknowledged</b>              | 1  |

## 2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

### 2.1 Scope

The following contracts are in scope:

```
src/bridging/BaseBridgeAdapter.sol
src/bridging/ccip/CcipAdapter.sol
src/core/Allocator.sol
src/core/BaseChainGateway.sol
src/core/LocalBalanceAggregator.sol
src/core/accounting/AccountingChainGateway.sol
src/core/accounting/FundsHandler.sol
src/core/accounting/StableVault.sol
src/core/earning/EarningChainGateway.sol
src/core/ious/IouToken.sol
src/core/ious/IouTokenManager.sol
src/interfaces/IAccountingChainGateway.sol
src/interfaces/IAllocator.sol
src/interfaces/IAssetRegistry.sol
src/interfaces/IBridgeAdapter.sol
src/interfaces/ICcipBridgeAdapter.sol
src/interfaces/IChainBalanceOracle.sol
src/interfaces/IChainBalanceOracleAdapter.sol
src/interfaces/IChainGateway.sol
src/interfaces/IEarningChainGateway.sol
src/interfaces/IEarningChainStateProvider.sol
src/interfaces/IFundsHandler.sol
src/interfaces/IIouToken.sol
src/interfaces/IIouTokenManager.sol
src/interfaces/IMintableBurnableIERC20.sol
src/interfaces/IMulticall.sol
src/interfaces/IPriceOracle.sol
src/interfaces/IPriceOracleAdapter.sol
src/interfaces/IRescuableNative.sol
src/interfaces/IRescuableToken.sol
src/interfaces/ISTableVault.sol
src/interfaces/ISwapper.sol
src/interfaces/ITransferHelper.sol
src/interfaces/IWithdrawalPolicy.sol
src/libraries/AssetLib.sol
src/libraries/MathLib.sol
src/misc/Multicall.sol
src/misc/RescuableNative.sol
src/misc/RescuableToken.sol
src/misc/TransferHelperClient.sol
src/oracles/balance/ChainBalanceOracle.sol
src/oracles/balance/ChainlinkChainBalanceOracleAdapter.sol
src/oracles/balance/ChainlinkL2ChainBalanceOracleAdapter.sol
```

```
src/oracles/common/L2ChainlinkOracleAdapter.sol
src/oracles/price/ChainlinkL2PriceOracleAdapter.sol
src/oracles/price/ChainlinkPriceOracleAdapter.sol
src/oracles/price/PriceOracle.sol
src/periphery/AssetRegistry.sol
src/periphery/EarningChainStateProvider.sol
src/periphery/EarningChainStateSchemaV1.sol
src/periphery/OwnedMulticall.sol
src/periphery/Swapper.sol
src/periphery/TransferHelper.sol
src/periphery/WithdrawalPolicy.sol
src/types/Constants.sol
src/types/Errors.sol
```

In version 2 of the review, the scope was expanded to cover the AccessManager role configuration scripts, verified against the Stable Vault Roles Spreadsheet shared by the client. This covers the relevant code in the following files:

```
script/base/RolesConfig.sol
script/base/AccessManagerBaseSetup.sol
script/base/AccessManagerAccountingChainSetup.sol
script/base/AccessManagerEarningChainSetup.sol
```

Changes to the main codebase introduced in version 2 were reviewed in the third iteration (fixes review).

The assessment was performed on the source code files inside the Aave Stable Vaults repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

| V | Date             | Commit Hash                              | Note                     |
|---|------------------|------------------------------------------|--------------------------|
| 1 | 25 February 2026 | afe05e68a66f299e61a2f840937765c116bbd86c | Initial Version          |
| 2 | 19 March 2026    | c27a4a01f5059942091fb793677df07c4d368a39 | Roles Setup Review       |
| 3 | 04 May 2026      | dcf3d30ff91501c80293434d331597cb0e065f00 | Chainsecurity Fix Review |

For the solidity smart contracts, the compiler version 0.8.28 and evm version cancun were chosen.

### 2.1.1 Excluded from scope

Any file not explicitly listed in scope is excluded from this review. In particular:

- Deployment and configuration scripts
- Test files
- Third-party dependencies (OpenZeppelin, Chainlink contracts)

Additionally, gas optimizations are excluded from this assessment.

## 2.2 Assumptions

This assessment assumes the validity of the assumptions documented by the client in the StableVault specifications. Issues arising from violations of these assumptions are outside the scope of this audit. Key assumptions include:

- All whitelisted ERC-4626 strategies are audited and trusted
- Admin maintains configuration consistency across Accounting and Earning chains
- Chainlink infrastructure is available and accurate

## 2.3 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Aave Labs offers Aave Stable Vaults, a cross-chain semi-fixed-rate stablecoin vault. Users deposit stablecoins on the Accounting Chain (Arbitrum) and earn a predetermined per-second interest rate, while protocol managers deploy those assets into ERC-4626 yield strategies on the Accounting Chain, with optional deployment to Earning Chains such as Ethereum mainnet. Withdrawals are two-phase: users first request a withdrawal that converts their vault position into IOU tokens, then execute the withdrawal by burning IOUs for underlying assets, subject to a configurable fee. The key actors are depositors, IOU holders, protocol managers who rebalance and bridge funds, a governance layer backed by OpenZeppelin AccessManager with timelocked and immediate execution tiers, and Chainlink oracle infrastructure that relays price and balance data across chains.

### 2.3.1 StableVault

StableVault is the user-facing entry point on the Accounting Chain. It implements a semi-fixed-rate vault where each interest rate tier is modeled as a *sub-vault* with its own per-second conversion rate and share supply. When a user deposits, their assets are routed through FundsHandler into the Allocator. In return, the user receives shares in their assigned sub-vault. The conversion rate accrues continuously:  $\text{conversionRate} *= \text{perSecondRate} \wedge \text{elapsed}$ . A user's balance in RAY terms equals  $\text{shares} * \text{conversionRate}$ . Each user holds at most one position in one sub-vault, where the governance can move users between sub-vaults or change sub-vault rates.

The withdrawal flow is two-phase. In `requestWithdrawal()`, shares are burned and the vault computes how much of the withdrawal is *guaranteed* (backed by the user's original deposit principal) versus *interest* (which must be covered by the system's surplus assets over obligations). The function enforces a global solvency check, where the interest portion of the request must not exceed  $\text{totalAssets} - \text{guaranteedObligations}$ , where `totalAssets` aggregates local Allocator balances and the latest Earning Chain balance snapshots from the ChainBalanceOracle. Upon passing the check, IOU tokens are minted to the user. In `executeWithdrawal()`, the user burns IOUs, where withdrawal fees are applied through `WithdrawalPolicy`. Fees are taken in form of IOU tokens and burned, reducing the system's guaranteed obligations. The net amount is withdrawn from the Allocator through FundsHandler and transferred to the user.

Withdrawals consume the guaranteed (principal) portion of a user's balance first. The amount exceeding the original deposit is treated as interest and subject to the solvency check. Users can always request IOUs corresponding to their original deposits. However, when the system's surplus is insufficient to cover all accrued interest, interest withdrawals follow first-come-first-served order. Earlier withdrawers claim their interest as IOUs, while later users may be limited to their guaranteed portion.

Similarly, if the vault becomes underfunded, IOU redemption in `executeWithdrawal()` and `EarningChainGateway.exchangeIouTokens()` is first-come-first-served with respect to the assets available in the (local) Allocator.

The vault also supports `transfer()` and `transferAll()` to move positions between users (accounting-only, no asset movement), and `claimSurplusInterest()` which lets governance withdraw earned yield above total obligations to a treasury address.

The contract provides the following functions:

- `deposit(user, asset, amount)`: Permissionless. Deposits a whitelisted stablecoin for the specified user. Validates the asset price via `PriceOracle.validatePrice()`, routes funds through `FundsHandler` into the `Allocator`, and mints sub-vault shares. Reverts if the asset's price is below the minimum valid price or if deposits are disabled for that asset.
- `requestWithdrawal(user, requestedAmountInRay)`: Only callable by the user themselves. Burns sub-vault shares and mints IOU tokens. Passing 0 triggers a full withdrawal. Enforces the global solvency invariant before minting IOUs.
- `executeWithdrawal(user, assetOut, minAmountOut, iouAmountRay, data)`: Only callable by the user themselves. Burns IOUs, applies the withdrawal fee via `WithdrawalPolicy`, withdraws assets from `Allocator`, and transfers them to the user.
- `transfer(to, amountRay)`: Transfers a portion of the caller's vault position to another user. Accounting-only.
- `transferAll(to)`: Transfers the caller's entire position (shares and original deposit tracking) to another user. Accounting-only.
- `setUserRate(userRateData[])`: Restricted (immediate). Moves users to sub-vaults matching new per-second rates, creating sub-vaults as needed.
- `setSubVaultRate(subVaultId, newPerSecondRate)`: Restricted (immediate). Changes the interest rate of an existing sub-vault. Accrues before updating.
- `setDefaultSubVault(perSecondRate)`: Restricted (immediate). Sets the default sub-vault assigned to new depositors.
- `claimSurplusInterest/assets[], amounts[]`: Restricted (7-day delay). Withdraws assets from the `Allocator` up to the surplus above total obligations, and sends them to the treasury.
- `setTreasury(treasury)`: Restricted (7-day delay). Sets the treasury address for surplus claims.

## 2.3.2 FundsHandler

`FundsHandler` acts as the intermediary between the `StableVault` and the `Allocator` on the Accounting Chain. It routes deposit and withdrawal requests to the local `Allocator` and computes the system-wide aggregated balance by summing the local `Allocator` balance (via `LocalBalanceAggregator`) with Earning Chain balance snapshots from the `ChainBalanceOracle`. Stale Earning Chain balances are treated as zero, deliberately under-estimating the total asset value to prevent the system from over-committing obligations against assets it cannot confirm.

The contract also manages the set of registered Earning Chain IDs and provides the `pushFundsToChain()` function for managers to bridge assets from the Accounting Chain to an Earning Chain via the `AccountingChainGateway`. When assets arrive from an Earning Chain (via the gateway's `fundsArrivedFromChainCallback`), they are deposited into the local `Allocator` using `depositAllowIdle()`, which tolerates strategy deposit failures gracefully.

The contract provides the following functions:

- `processDeposit(asset, amount)`: Only callable by `StableVault`. Deposits assets into the `Allocator`.
- `processWithdrawal(asset, amount)`: Only callable by `StableVault`. Withdraws assets from the `Allocator`.
- `getAggregatedBalance()`: View. Returns the total system balance in RAY. Computes the local `Allocator` balance plus all non-stale Earning Chain balances.

- `pushFundsToChain(asset, amount, chainId, bridgeParams)`: Restricted (immediate). Withdraws assets from the Allocator and bridges them to a registered Earning Chain.
- `fundsArrivedFromChainCallback(asset, amount)`: Only callable by the AccountingChainGateway. Deposits received assets into the Allocator.
- `addEarningChain(chainId) / removeEarningChain(chainId)`: Restricted (7-day delay). Manages the set of Earning Chains whose balances are included in the aggregated balance.

### 2.3.3 Allocator

Allocator manages the deployment of stablecoin assets into ERC-4626 yield strategies. Each registered asset can have up to `MAX_STRATEGIES_PER_ASSET` strategies, one of which is designated as the *default* strategy. On deposit, assets are routed to the default strategy. On withdrawal, a cascading logic first consumes idle balance held directly by the Allocator, then withdraws from the default strategy, then iterates through remaining strategies until the requested amount is fulfilled. Each strategy interaction is wrapped in a try/catch so that a single reverting strategy does not block the entire withdrawal.

Strategy deposits use a low-level call to `IERC4626.deposit()` and verify the balance change is within a configured `maxSlippageAmount`. The `rebalance()` function allows managers to execute an ordered sequence of deallocations, swaps (via an external Swapper), and allocations in a single transaction.

Deposits into and withdrawals from the Allocator are restricted to a single whitelisted `DEPOSITOR` and `WITHDRAWER` address respectively, set immutably at construction time (FundsHandler on the Accounting Chain, EarningChainGateway on Earning Chains).

The contract provides the following functions:

- `deposit(asset, amount)`: Only callable by the depositor. Deposits into the default strategy. Leaves assets idle if no default strategy is set. Reverts if strategy deposit fails.
- `depositAllowIdle(asset, amount)`: Only callable by the depositor. Same as `deposit()` but leaves assets idle if the strategy deposit reverts.
- `withdraw(asset, amount)`: Only callable by the withdrawer. Reverts if the full amount cannot be sourced.
- `rebalance(params[])`: Restricted (immediate). Executes ordered deallocations, swaps, and allocations.
- `addStrategy(asset, strategy, maxSlippageAmount)`: Restricted (7-day delay). Registers a new ERC-4626 strategy for an asset.
- `removeStrategy(strategy)`: Restricted (immediate). Removes a strategy. Requires zero balance in the strategy.
- `setDefaultStrategy(asset, strategy)`: Restricted (immediate). Sets the default deposit target for an asset.
- `enableDepositsToStrategy(strategy) / disableDepositsToStrategy(strategy)`: Restricted (7-day delay / immediate respectively). Toggles deposit permission per strategy.

### 2.3.4 AccountingChainGateway

AccountingChainGateway extends BaseChainGateway and serves as the cross-chain messaging hub on the Accounting Chain. It handles three inbound message types from Earning Chains:

- `BRIDGE_IOU_TOKEN`: Releases locked IOUs to a recipient via the IouTokenManager.
- `BURN_IOU_TOKEN`: Burns locked IOUs to synchronize obligations after an Earning Chain IOU exchange.

- **RETURN\_FUNDS:** Validates the inbound message's block number against the ChainBalanceOracle snapshot to prevent double-counting of returning assets.

For outbound communication, Accounting Chain gateway sends asset-bearing messages to Earning Chains on behalf of FundsHandler.

The `_validateInboundMessageBlockNumber()` check is a critical consistency mechanism. It rejects inbound `BURN_IOU_TOKEN` and `RETURN_FUNDS` messages whose source block number is newer than the latest oracle snapshot, preventing the Accounting Chain from decrementing obligations or accepting funds based on state that its balance oracle has not yet captured.

The contract provides the following functions:

- `sendPushFundsToChainMessage(asset, amount, targetChainId, bridgeParams):` Only callable by FundsHandler. Sends assets to an Earning Chain via the default bridge adapter.
- `receiveMessage(sourceChainId, asset, amount, data):` Called by bridge adapters to deliver cross-chain messages. Funds reception does not require adapter whitelisting, whereas data processing does.

### 2.3.5 EarningChainGateway

EarningChainGateway extends BaseChainGateway and is deployed on each Earning Chain. It communicates with the Accounting Chain. Its primary user-facing function is `exchangeIouTokens()`, which allows IOU holders to burn their IOUs and receive underlying stablecoin assets from the local Allocator, subject to the withdrawal fee. This enables users to exit the system on any Earning Chain that holds liquidity, not just on the Accounting Chain.

When assets are received from the Accounting Chain, they are deposited into the local Allocator using `depositAllowIdle()`. Managers can return assets to the Accounting Chain via `pushFundsToAccountingChain()`. On IOU exchange, the gateway sends a `BURN_IOU_TOKEN` message to the Accounting Chain.

The contract provides the following functions:

- `exchangeIouTokens(iouTokenAmountRay, assetOut, minAmountOut, receiver, bridgeParams, data):` Permissionless. Burns the caller's IOUs, applies withdrawal fee, withdraws assets from Allocator, sends a `BURN_IOU_TOKEN` message to the Accounting Chain, and transfers assets to the specified receiver.
- `pushFundsToAccountingChain(asset, amount, bridgeParams):` Restricted (immediate). Withdraws from the local Allocator and bridges assets plus a `RETURN_FUNDS` message to the Accounting Chain.
- `getAggregatedBalance():` View. Returns the local Allocator balance in RAY.

### 2.3.6 BaseChainGateway

BaseChainGateway is the abstract base for both gateway contracts. It manages bridge adapter whitelisting: per-asset, per-chain-ID mappings of supported adapters and a default adapter for outbound messages. The `receiveMessage()` function implements a split path: fund reception (token transfers from bridge) does not require the calling adapter to be whitelisted (allowing recovery of stuck tokens even from removed adapters), while data processing (cross-chain messages) strictly requires a whitelisted adapter.

The contract provides the following functions:

- `addBridgeAdapter(asset, chainId, adapter):` Restricted (7-day delay). Whitelists a new bridge adapter.
- `removeBridgeAdapter(asset, chainId, adapter):` Restricted (immediate). Removes a bridge adapter. Also clears it from the default if applicable.

- `setDefaultBridgeAdapter(asset, chainId, adapter)`: Restricted (immediate). Sets the adapter used for outbound messages.
- `sendBridgeIouTokenMessageWithFeePayer(destChainId, recipient, amount, bridgeParams)`: Only callable by `IouTokenManager`. Publishes IOU bridge messages to a destination chain.
- `receiveMessage(sourceChainId, asset, amount, data)`: Called by bridge adapters. Routes funds and data to the concrete gateway implementation.

## 2.3.7 *IouTokenManager*

`IouTokenManager` manages the lifecycle of IOU tokens. On the Accounting Chain, IOUs are minted by `StableVault` during `requestWithdrawal()` and can be *locked* when a user bridges them to an Earning Chain. When the Earning Chain confirms a burn (via `BURN_IOU_TOKEN` message), the locked tokens are burned. When an Earning Chain bridges IOUs back, locked tokens are released. On Earning Chains, the manager mints IOUs on receipt of a bridge message and burns them during `exchangeIouTokens()`.

The contract provides the following functions:

- `bridgeTokens(destinationChainId, recipient, amount, bridgeParams)`: Permissionless. Locks IOUs (Accounting Chain) or burns IOUs (Earning Chain) and sends a bridge message.
- `mintTokens(to, amount) / burnTokens(from, amount)`: Only callable by the Vault or Gateway.
- `burnLockedTokens(amount)`: Only callable by the Gateway on the Accounting Chain. Burns previously locked IOUs.
- `releaseTokens(to, amount)`: Only callable by the Gateway on the Accounting Chain. Releases locked IOUs to a recipient.

## 2.3.8 *IouToken*

`IouToken` is a minimal ERC-20 token with 27 decimals (RAY precision) owned by `IouTokenManager`. It exposes `mint()`, `burn()`, and `lock()` functions restricted to the owner. The `lock()` function transfers tokens from a user to the manager contract, effectively escrowing them during cross-chain bridging.

## 2.3.9 *CcipAdapter*

The current implementation uses Chainlink CCIP, but the architecture supports adding alternative bridge adapters. `CcipAdapter` is the Chainlink CCIP bridge adapter, extending `BaseBridgeAdapter`. It translates between the system's chain IDs and CCIP chain selectors, constructs `EVM2AnyMessage` structs, and handles fee payment and refunds. Inbound messages (`ccipReceive()`) are validated against the registered source chain adapter and then routed. Data is processed first (allowing state modifications before token handling), followed by token processing via `_processReceivedFunds()`.

The adapter includes `replayFundsReceiving()` for recovering registered assets that were directly sent to the adapter. Registered assets stuck in the adapter can only be recovered through this replay mechanism. The function `rescueTokens` is blocked for registered assets as a safeguard.

The contract provides the following functions:

- `publishMessageToChainWithFeePayer(destChainId, asset, amount, data, bridgeParams)`: Only callable by the Gateway. Constructs and sends a CCIP message with token and/or data payloads.
- `ccipReceive(message)`: Only callable by the CCIP Router. Validates source, processes data then tokens.

- `replayFundsReceiving(asset, amount)`: Restricted (immediate). Re-processes stuck registered assets.
- `setChainSelector(chainId, ccipChainSelector)`: Restricted (7-day delay). Maps chain IDs to CCIP selectors.
- `setDestinationChainAdapter(chainId, adapter)`: Restricted (7-day delay). Inherited from `BaseBridgeAdapter`. Sets the expected remote adapter address.

### 2.3.10 PriceOracle

`PriceOracle` is the registry contract that provides asset prices through pluggable adapter contracts. It enforces a ceiling of 1 RAY (1e27) on returned prices, reflecting the system's assumption that supported assets are stablecoins pegged at or near 1:1 with the denomination unit. A configurable `MIN_VALID_PRICE_RAY` (deployed as 0.997 RAY) is used by `validatePrice()` to gate deposits. Assets whose oracle price has fallen below the minimum are blocked from entering the vault.

When an adapter reports a stale price, `getPrice()` returns 0 rather than reverting, causing `LocalBalanceAggregator` to under-value the affected asset. The `validatePrice()` function, used during deposits, reverts on stale prices, preventing deposits of depegged or unpriced assets.

The contract provides the following functions:

- `getPrice(asset)`: View. Returns the price in RAY, capped at 1 RAY. Returns 0 if stale.
- `validatePrice(asset)`: View. Reverts if the price is stale or below `MIN_VALID_PRICE_RAY`. Used to gate deposits.
- `setOracleAdapterForAsset(asset, adapter)`: Restricted (7-day delay). Sets the price feed adapter for an asset.

### 2.3.11 ChainBalanceOracle

`ChainBalanceOracle` is the registry contract on the Accounting Chain and provides Earning Chain aggregated balance values through pluggable adapter contracts (one per Earning Chain). The adapter call is wrapped in a try/catch. If the adapter reverts, a zero-balance stale response is returned rather than propagating the failure, ensuring the consuming contracts (`FundsHandler.getAggregatedBalance()`) are not disrupted.

The contract provides the following functions:

- `getChainBalance(chainId)`: View. Returns the balance snapshot for an Earning Chain including the source chain block number and staleness status.
- `setChainBalanceOracleAdapter(chainId, adapter)`: Restricted (7-day delay). Sets the balance feed adapter for an Earning Chain.

### 2.3.12 Oracle Adapters

`ChainlinkPriceOracleAdapter` queries a `Chainlink AggregatorV3Interface` data feed and converts the result to RAY precision. It marks the response as stale if the feed has not been updated within the last `HEARTBEAT + 90s`. Negative or zero prices are returned as 0.

`ChainlinkChainBalanceOracleAdapter` queries a `Chainlink Bundle Aggregator` that publishes `EarningChainStateProvider.getState()` data. It validates the schema version and chain ID within the snapshot, and marks the response as stale using the same heartbeat pattern.

Both adapters have L2 variants (`ChainlinkL2PriceOracleAdapter`, `ChainlinkL2ChainBalanceOracleAdapter`) that extend the base adapters with a Chainlink L2 Sequencer Uptime Feed check. If the sequencer is down or has been up for less than the 1 hour grace period, all responses are marked as stale.

Beyond the heartbeat, Aave Labs states that the Chainlink network runs a deviation check at intervals shorter than a minute and publishes a new update to the feed on the Accounting Chain whenever the reported value moves beyond a configured threshold. In practice the feeds are therefore refreshed well within the heartbeat window.

EarningChainStateProvider is deployed on each Earning Chain and exposes the local Allocator's aggregated balance (via `EarningChainGateway.getAggregatedBalance()`). A Chainlink oracle network reads this on-chain and publishes it to the Accounting Chain's bundle aggregator.

### 2.3.13 Periphery Contracts

AssetRegistry manages registered assets and their configuration flags (`depositFromUserAllowed`, `depositIntoAllocatorAllowed`, `swapInputTokenAllowed`, `swapOutputTokenAllowed`). It also maintains a *trusted* set. Only trusted assets contribute to balance aggregation. All enabling operations have a 7-day delay, whereas disabling operations are immediate for emergency response. Assets are limited to 18 decimals for RAY precision compatibility.

WithdrawalPolicy enforces a fee on withdrawals (both `StableVault.executeWithdrawal()` and `EarningChainGateway.exchangeIouTokens()`). The fee is determined by three-tier priority:

1. An EIP-712 signed personal fee discount from a whitelisted signer
2. An asset-specific fee
3. A global default fee

All fees are capped at 5% (500 bps) and rounded up, favoring the protocol. The signed discount mechanism uses per-signer nonces with deadlines for replay protection.

Swapper is owned by the Allocator and executes arbitrary swap logic during rebalancing via an array of calls to external targets (e.g. DEX routers). It enforces a 1:1 conversion between input and output assets (adjusted for decimals), with shortfalls covered from a designated `slippageCoverageSource` up to a configured tolerance.

TransferHelper is a stateless, permissionless intermediary that holds assets temporarily during multi-step flows, enabling non-adjacent contracts to transfer assets without direct approvals. The modifier `assertingTransferHelperBalanceFor` in `TransferHelperClient` is used to ensure that the helper's balance does not increase by the end of the transaction.

OwnedMulticall is a privileged batching proxy that lets the owner execute multiple protocol actions atomically. The deployed contract holds protocol roles, allowing the owner to batch token approvals, slippage coverage setup, and rebalance calls. The owner has unrestricted ability to call any address with any calldata through the aggregate functions with no on-chain target whitelisting. The contract can also act as the Swapper's slippage coverage source; tokens held have no on-chain cap or accounting.

### 2.3.14 Trust Model

**Operator:** Users deposit assets into a managed vault where operators control strategy selection, cross-chain fund distribution, and interest rate assignments. Operators can move users to different sub-vaults or change sub-vault rates without delay, affecting future interest accrual. While the system provides safeguards (timelocked administrative actions for strategy and adapter changes, guaranteed principal accounting that limits interest claims to available surplus, ordered redemption, and cross-chain IOU bridging to access liquidity on other chains), these protections rely on underlying liquidity being available. In case of losses from strategy failures or asset depeg, users are implicitly relying on operators to restore solvency.

**Governance (AccessManager admin / High-Threshold Multisig):** Privileged, but the system is designed to be non custodial: every action capable of misappropriating user funds is subject to an execution delay, giving users a monitoring and exit window before the action takes effect. Within that constraint, there are delayed pathways through which a malicious governance could ultimately divert funds:

1. `claimSurplusInterest()` after manipulating balance oracle adapters to inflate `totalAssets`. (7-day delay)
2. Upgrading contracts to malicious implementations. (all core contracts are upgradeable via proxy, 14-day delay)
3. Adding a malicious strategy via `addStrategy()` that siphons deposited assets. (7-day delay)
4. Adding a malicious bridge adapter that intercepts bridged funds. (7-day delay)

The `AccessManager` uses a two-tier guardian hierarchy (`ADMIN_ROLE_GUARDIAN_ROLE` and `OPERATIONAL_ROLE_GUARDIAN_ROLE`) that enables emergency role revocation without privilege escalation. Execution delays are organized into three tiers: critical delay for `AccessManager` admin operations and proxy upgrades, medium delay for sensitive configuration changes, and no delay for operational and emergency functions.

**Manager (Rebalancer role):** Partially trusted. Can execute `rebalance()` on the `Allocator` which includes arbitrary external calls via the `Swapper` (to DEX routers etc.). Swaps enforce 1:1 conversion with slippage bounds, and only whitelisted swap-eligible assets can be swapped. Can call `pushFundsToChain()` and `pushFundsToAccountingChain()` to bridge assets between chains, but only to registered Earning Chains and through whitelisted bridge adapters. Cannot directly withdraw assets to arbitrary addresses. A compromised manager could cause temporary liquidity shortages by misallocating funds across chains or concentrating assets in underperforming strategies.

**Withdrawal Fee Signers:** Partially trusted. Can sign EIP-712 fee discount messages that reduce or eliminate the withdrawal fee for specific users. Cannot steal funds directly. A compromised signer can issue zero-fee withdrawals, eliminating the protocol's fee revenue and enabling costless arbitrage against the vault.

**End Users:** Untrusted. Deposits are gated by `AssetRegistry` whitelisting and oracle price checks. Withdrawals enforce fees and solvency checks. IOU bridging uses lock-on-source/mint-on-destination with finality assumptions. Users can specify an arbitrary receiver for `exchangeIouTokens()` on Earning Chains, but this only directs their own withdrawn assets. Users pay bridge fees and cannot grief the protocol without economic cost (withdrawal fees, gas). Users are expected to be aware of the system's non custodial design: privileged actions that could affect the system's configuration and hence access to funds are gated by execution delays, and users are expected to withdraw within those exit windows in case of unexpected or unfavorable changes.

**Chainlink Oracles (Price + Balance):** The system relies on Chainlink price feeds for deposit gating, balance valuation, and swap validation. A manipulated or stale price feed could:

1. Allow deposits of a depegged asset at inflated value.
2. Undervalue assets causing the system to appear insolvent and block interest withdrawals.

The Chainlink balance oracle relays Earning Chain balances to the Accounting Chain. A stale balance feed causes the system to treat the Earning Chain balance as zero, underestimating total assets.

**CCIP Bridge:** The system relies on Chainlink CCIP for cross-chain message and token delivery. Message authentication is enforced by validating the source chain adapter address against the registered `_destinationChainAdapterOf` mapping. A failure or compromise of CCIP could:

1. Delay or prevent cross-chain fund movements and IOU bridging.
2. If message forgery were possible (requires CCIP-level compromise), allow minting of unbacked IOUs on Earning Chains or burning of IOUs without corresponding asset release.

The adapter does not implement a defensive receiver pattern.

**ERC-4626 Strategies:** Trusted. Each strategy is whitelisted with a 7-day delay and has a `maxSlippageAmount` bound. A malicious or buggy strategy can cause fund loss. The system assumes:

- Strategies are audited and vetted before whitelisting (7-day delay)
- Strategies do not charge withdrawal fees (required for accurate `previewRedeem()` accounting)



- Strategies are not susceptible to inflation attacks
- Off-chain monitoring continuously tracks strategy health and reduces exposure if risk tolerance is breached
- If a strategy is compromised, deposits are disabled immediately

If a strategy violates these assumptions, the system can lose funds. Inflation attacks are particularly dangerous at deposit time: if a strategy is inflated when assets are deposited, the system receives fewer shares than the real value while crediting users with full `originalDepositRay`. Beyond inflation, a strategy could return fewer assets than expected on withdrawal (bounded by per-strategy `maxSlippageAmount`) or lose deposited funds entirely through DeFi risk.

**Upgradeability:** All core contracts (`StableVault`, `FundsHandler`, `Allocator`, `AccountingChainGateway`, `EarningChainGateway`, `AssetRegistry`, `WithdrawalPolicy`, `PriceOracle`, `ChainBalanceOracle`) are upgradeable via `AccessManagedUpgradeable`. Bridge adapters (`CcipAdapter`) and utility contracts (`Swapper`, `TransferHelper`, `OwnedMulticall`, `IouToken`, `IouTokenManager`) are not upgradeable. The upgrade authority rests with the `AccessManager` admin, subject to the critical delay (14 days). Once executed, a malicious upgrade can bypass all system invariants.

### 2.3.15 Operational Assumptions

Malicious or compromised manager accounts are expected to be revoked within 1 hour of detection via guardian action, enabled by off-chain monitoring. The system assumes correct cross-chain configuration: oracle adapters, bridge adapters, and chain registrations must be configured consistently across all chains.

### 2.3.16 Appendix: Roles Setup

*This appendix documents the roles configuration of **Version 2**.*

The system uses OpenZeppelin's `AccessManager` as the central access control registry, deployed independently on each chain. All contracts inheriting `AccessManaged` reference their chain's manager, which maps function selectors and targets to required roles and enforces execution delays.

#### Role Hierarchy

Three special roles govern access control:

- `ADMIN_ROLE`: Super admin with 14-day execution delay. Can call any `AccessManager` function including proxy upgrades.
- `ADMIN_ROLE_GUARDIAN_ROLE`: Can grant admin tier function roles (7-day delay) and revoke them immediately.
- `OPERATIONAL_ROLE_GUARDIAN_ROLE`: Can grant/revoke operational function roles (immediate, non-critical).

The guardian roles enable emergency revocation.

#### Function Roles

The system defines function based roles derived from selectors. Each specifies an execution delay and guardian assignment. Seven functions are marked *critical-risk* (can directly compromise funds): `setAssetConfig`, `addBridgeAdapter`, `setDestinationChainAdapter`, `setChainSelector`, `setTreasury`, `addStrategy`, `setChainBalanceOracleAdapter`.

#### Delay Tiers

- *Critical (14 days)*: `ADMIN_ROLE` operations and proxy upgrades.
- *Medium (7 days)*: Configuration changes enabling fund movement (adding strategies, bridges, oracles, enabling deposits).

- *Immediate*: Operational and emergency functions (rebalancing, disabling, rescue).

Disabling operations are immediate for emergency response.

## Profiles

Profiles are predefined bundles of roles.

- *MainAdmin*: ADMIN\_ROLE + both guardian roles + all function roles. Full control with delays.
- *SecondaryAdmin*: OPERATIONAL\_ROLE\_GUARDIAN\_ROLE + non-critical function roles. Cannot add strategies or bridges.
- *StableVaultManager* (Accounting Chain only): Interest rate management functions.
- *WithdrawalPolicyManager*: Fee configuration.
- *Rebalancer*: Fund movement via `rebalance()`, `pushFundsToChain()`, strategy selection.
- *Disabler*: Emergency shutdown (rescue, remove, disable, distrust functions).

Rebalancer and Disabler operate through `OwnedMulticall` contracts for atomic batching.

## Proxy Upgrades

All upgradeable contracts use `TransparentUpgradeableProxy` with proxy admin owned by `AccessManager`. Upgrades require ADMIN\_ROLE (14-day delay).

## 2.3.17 Version 3 Changes

*This appendix notes functional additions in **Version 3**.*

`Allocator.topUp(asset, amount)` is a new restricted entry point that lets a privileged caller pull a registered asset into the `Allocator` to improve solvency. The amount is added to the idle balance and `AssetToppedUp` is emitted.

Strategies can now be marked as trusted or untrusted. `trustStrategy` and `distrustStrategy` change the state, with `distrustStrategy` immediate and `trustStrategy` subject to the medium delay. Balance computations used for solvency checks include only trusted strategies, while the general balance query includes all strategies.

### 3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

## 4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

| Likelihood | Impact   |        |        |
|------------|----------|--------|--------|
|            | High     | Medium | Low    |
| High       | Critical | High   | Medium |
| Medium     | High     | Medium | Low    |
| Low        | Medium   | Low    | Low    |

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

# 5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the [Resolved Findings](#) section. The findings are split into these different categories:

- **Design**: Architectural shortcomings and design inefficiencies
- **Correctness**: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

|                                    |   |
|------------------------------------|---|
| <b>Critical</b> -Severity Findings | 0 |
| <b>High</b> -Severity Findings     | 0 |
| <b>Medium</b> -Severity Findings   | 0 |
| <b>Low</b> -Severity Findings      | 2 |

- [Weakened Adapter Registration Check in ChainBalanceOracle](#) **Acknowledged**
- [Contradicting Trust Assumption on Admin Role](#) **Risk Accepted**

## 5.1 Weakened Adapter Registration Check in ChainBalanceOracle

**Correctness** **Low** **Version 3** **Acknowledged**

CS-ASV-001

`ChainBalanceOracle.setChainBalanceOracleAdapter()` validates the new adapter by calling `supportsInterface(type(IChainBalanceOracleAdapter).interfaceId)`. The returned `bool` is not checked, so an adapter that responds to the selector with `false` is still accepted. The check therefore only verifies that the new adapter exposes a `supportsInterface` function that does not revert, not that the new adapter actually claims support for the expected interface.

The earlier version of this function called `getChainBalance(chainId)` on the adapter. The adapter's implementation requires `chainId == EARNING_CHAIN_ID` and reverts otherwise, so the registration call implicitly enforced that the adapter was configured for the chain it was being registered under. This implicit check is lost with the switch to `supportsInterface`.

`PriceOracle.setOracleAdapterForAsset()` remains unchanged and uses the original pattern of calling the data query function (`getPrice(asset)`) rather than `supportsInterface`, so the two oracle setters now use diverging validation strategies.

### Acknowledged:

Aave Labs states:

The divergence from `PriceOracle.setOracleAdapterForAsset` is intentional: the price oracle's `getPrice(asset)` query is live at deploy time, so the data-query pattern works there. The chain-balance bundle feed is deployed-but-not-yet-streaming until Chainlink enables the workflow post-deploy, so a strict `getChainBalance(chainId)` probe would revert during deployment.

Implicit `chainId == EARNING_CHAIN_ID` enforcement is preserved at the adapter use sites (`getChainBalance` and the snapshot decode revert on mismatch). Setter is admin-gated.

## 5.2 Contradicting Trust Assumption on Admin Role

**Correctness** **Low** **Version 2** **Risk Accepted**

CS-ASV-002

`access_manager_setup.md` states (lines 19-22):

- All delays and security mechanisms exist to protect against compromised operational keys, not compromised ADMIN\_ROLE
- ADMIN\_ROLE has a 14-day execution delay, providing a time window for emergency response

This is contradictory. An admin compromise cannot be protected against, as correctly stated in the first statement. The second statement does not hold: there is not necessarily a time window for emergency response. Consider the following scenario:

If the main admin role is compromised on both chains, it can immediately revoke all other roles on both chains (line 236: "guardian roles have 0 execution delay") and schedule adding new malicious strategies for all assets. This has a time delay of 7 days, not 14 (line 258: `addStrategy` is `MED_DELAY`), so funds can be extracted after 7 days already, not 14 days.

During this time users could theoretically withdraw. However, a compromised admin can immediately grant itself operational roles, rebalance and push funds to earning chains (assuming little redeemable IOUs there), then remove the bridge adapter (0 execution delay) blocking the path for users to bring their IOUs onto the chain with liquidity for actual withdrawal.

After the time delay is over funds are still available and the attacker can exit (after 7 days via malicious strategy, after 14 days via proxy update).

Note: unless MainAdmin signer sets are fully independent across chains, an admin compromise on one chain may also affect the other.

---

### Risk Accepted:

Client states:

The timelock enforced on transactions executed by the primary admin are not a defense mechanism against a compromise of the primary admin. Timelocks are set by severity and potential impact to users. Scheduled operations will be monitored by off-chain systems and inform users of imminent actions. Regarding the case outlined in the finding, the Chain Gateway contracts will not allow an admin to remove a bridge adapter to bridge IOUs via bridging arbitrary data if it is the only bridge adapter set to bridge arbitrary data. An additional clarification to make is that the admin itself can cancel scheduled operations; some subset of signers on the multisig can step in to cancel an operation which was scheduled and later determined to be undesirable to execute.

---

### Note:

This finding describes the system as reviewed in this report. The Chain Gateway behavior the Aave Labs refers to, separating funds adapters from data adapters and preventing removal of the last data adapter

for a chain (CannotRemoveLastDataOnlyBridgeAdapter), is part of the later version covered in the second report and was not present in the version reviewed here.

## 6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the [Open Findings](#) section.

Below we provide a numerical overview of the identified findings, split up by their severity.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <b>Critical</b> -Severity Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 0  |
| <b>High</b> -Severity Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 0  |
| <b>Medium</b> -Severity Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 2  |
| <ul style="list-style-type: none"><li>• <a href="#">Reverting previewRedeem Blocks Multiple System Flows</a> <b>Code Corrected</b></li><li>• <a href="#">User-Controlled gasLimit Can Cause BURN_IOU_TOKEN Message Failure</a> <b>Code Corrected</b></li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |    |
| <b>Low</b> -Severity Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 10 |
| <ul style="list-style-type: none"><li>• <a href="#">No Role for Immediate Signer Removal</a> <b>Code Corrected</b></li><li>• <a href="#">Distrusted Asset Deposits Not Blocked On-Chain</a> <b>Code Corrected</b></li><li>• <a href="#">Ignored from Parameter in Native Transfer Path</a> <b>Code Corrected</b></li><li>• <a href="#">Inaccessible Strategy Cannot Be Offboarded or Written Off</a> <b>Code Corrected</b></li><li>• <a href="#">Missing Reentrancy Guard on Allocator</a> <b>Code Corrected</b></li><li>• <a href="#">Reward Tokens May Become Stuck in Allocator</a> <b>Code Corrected</b></li><li>• <a href="#">Sequencer Grace Period May Be Insufficient Due to Arbitrum Timestamp Bounds</a> <b>Code Corrected</b></li><li>• <a href="#">Stale Surplus Snapshot in claimSurplusInterest</a> <b>Code Corrected</b></li><li>• <a href="#">Wrong Parameter Order in Event Emission</a> <b>Code Corrected</b></li><li>• <a href="#">CRITICAL_DELAY Discrepancy Between Documentation and Code</a> <b>Specification Changed</b></li></ul> |    |
| Informational Findings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 9  |
| <ul style="list-style-type: none"><li>• <a href="#">Code Inconsistencies and Readability</a> <b>Code Corrected</b></li><li>• <a href="#">Documentation Inconsistencies</a> <b>Specification Changed</b></li><li>• <a href="#">ERC4626 Strategy Rounding Loss on Dust Withdrawals</a> <b>Code Corrected</b></li><li>• <a href="#">Ineffective Payable Declarations</a> <b>Code Corrected</b></li><li>• <a href="#">Low-Level Call in _depositToStrategy</a> <b>Code Corrected</b></li><li>• <a href="#">Manager Operation Edge Cases</a> <b>Code Corrected</b> <b>Acknowledged</b></li><li>• <a href="#">Missing Getters and Setters for Configuration Mappings</a> <b>Code Corrected</b></li><li>• <a href="#">No Events Emitted on Configuration Updates and Administrative Actions</a> <b>Code Corrected</b></li><li>• <a href="#">Stale @custom:selector Annotations</a> <b>Code Corrected</b></li></ul>                                                                                                                                                |    |

### 6.1 Reverting previewRedeem Blocks Multiple System Flows

**Design** **Medium** **Version 1** **Code Corrected**



A single ERC-4626 strategy whose `previewRedeem` reverts causes a denial of service across deposits, withdrawal requests, balance reporting, surplus interest claims, and rebalancing.

`Allocator._getAssetBalanceInStrategy` calls `previewRedeem` without any revert protection:

```
function _getAssetBalanceInStrategy(IERC4626 strategy) internal view returns (uint256) {
    uint256 amount = strategy.previewRedeem(strategy.balanceOf(address(this)));
    return amount;
}
```

EIP-4626 specifies that `previewRedeem` "MAY revert due to other conditions that would also cause redeem to revert." A revert in any single strategy propagates uncaught through `_getTotalAssetBalance`, which iterates all registered strategies for a given asset, meaning one misbehaving strategy is sufficient to block the entire flow.

Notably, the codebase already handles reverting strategies elsewhere. `Allocator.withdraw` wraps every strategy interaction in `try this.tryWithdrawFromStrategy() ... catch`, and `depositAllowIdle` similarly uses `try this.tryDepositToStrategy() ... catch`.

The following flows revert if `previewRedeem` of any involved strategy reverts:

1. **Withdrawal requests.** `StableVault.requestWithdrawal` calls `_getVaultAggregatedBalance`, which reaches `_getTotalAssetBalance` through `FundsHandler.getAggregatedBalance` → `LocalBalanceAggregator._getLocalAggregatedBalance` → `Allocator.getTrustedAssetBalances`. All user withdrawal requests are blocked.
2. **Deposits.** `Allocator.deposit` calls `_depositToStrategy`, which invokes `_getAssetBalanceInStrategy` before and after the strategy deposit to measure slippage. All deposits into the affected asset are blocked.
3. **Surplus interest claims.** `StableVault.claimSurplusInterest` reads the vault's aggregated balance through the same `_getVaultAggregatedBalance` path.
4. **Earning chain balance reporting.** `EarningChainGateway.getAggregatedBalance` calls `_getLocalAggregatedBalance` through the same `LocalBalanceAggregator` path. This feeds into `EarningChainStateProvider.getState()`, which is consumed by the oracle network to report earning chain balances to the accounting chain. A revert causes stale or zero balance reporting. On the accounting chain this can cause `StableVault.requestWithdrawal` to revert with `InsufficientAssets` even when sufficient funds exist across chains.
5. **Rebalancing.** `Allocator.rebalance` calls `_depositToStrategy` for allocations, which uses `_getAssetBalanceInStrategy` for slippage measurement.

### Code corrected:

The new graceful `_tryGetAssetBalanceInStrategy()` (catches `previewRedeem()` reverts and returns 0) is used in view and aggregation flows (withdrawal requests, surplus claims, balance reporting), which are no longer `DoSable`. The slippage measurement in `_depositToStrategy()` retains the strict variant by design, since silently treating a revert as 0 would let the slippage check pass with a stale value. A deposit or rebalance that targets such a strategy will revert. The disabler role must call `distrustStrategy()` for the broken strategy to clear it as the default and restore the deposit flow.

## 6.2 User-Controlled gasLimit Can Cause BURN\_IOU\_TOKEN Message Failure

Design Medium Version 1 Code Corrected

CS-ASV-023

When a user calls `EarningChainGateway.exchangeIouTokens`, they supply the `bridgeParams` struct which includes the `gasLimit` for the CCIP message. This value is passed directly to the CCIP router in `CcipAdapter.publishMessageToChainWithFeePayer` without validation. The user pays the bridging fee for the BURN\_IOU\_TOKEN. Setting a low `gasLimit` reduces the CCIP fee quote, incentivizing users to minimize this value.

If the `gasLimit` is set too low, the BURN\_IOU\_TOKEN message reverts on the accounting chain due to out-of-gas. The user has already received their assets and the IOUs have been burned on the earning chain, but the accounting chain does not process the obligation reduction until the message is manually re-executed via the CCIP destination chain offramp.

---

### Code corrected:

`EarningChainGateway.exchangeIouTokens` now requires `bridgeParams.gasLimit >= MIN_BURN_IOU_TOKEN_GAS_LIMIT (120,000)`, preventing the BURN\_IOU\_TOKEN message from failing out-of-gas on the accounting chain.

## 6.3 No Role for Immediate Signer Removal

Design Low Version 2 Code Corrected

CS-ASV-014

There is a role for setting a signer. This function can only be called with a delay. However, there is no role configured that can disable a signer immediately. If a signer is compromised, the system cannot respond without waiting for the delay to pass.

---

### Code corrected:

`setSigner()` was split into `addSigner (MED_DELAY)` and `removeSigner()` (NO\_DELAY), allowing immediate revocation of a compromised signer.

## 6.4 Distrusted Asset Deposits Not Blocked On-Chain

Design Low Version 1 Code Corrected

CS-ASV-020

Trust status and `depositFromUserAllowed` are independent flags in the `AssetRegistry`. Calling `distrustAsset` does not disable `depositFromUserAllowed`. If an admin distrusts an asset without also disabling user deposits and/or allocator deposits, users can still deposit the asset. The deposited value is excluded from NAV (which only considers trusted assets) while the minted IOUs increase obligations, creating silent bad debt.

The `AssetRegistry` inherits `Multicall` which can be used for atomic distrust-and-disable operations, but no on-chain invariant enforces this. Disabling `depositIntoAllocatorAllowed` would also block

deposits by causing a revert when `Allocator.deposit` is reached, but this flag is likewise not automatically toggled by distrusting an asset.

The project documents this as Assumption #21 and intends to use atomic multicall operations to perform distrust-and-disable together. The invariant directly affects solvency accounting and is not enforced on-chain.

---

#### Code corrected:

`distrustAsset` now auto-disables `depositFromUserAllowed` and `swapOutputTokenAllowed`, blocking the user-facing deposit path that mints IOUs. `depositIntoAllocatorAllowed` remains a separate permission by design.

## 6.5 Ignored from Parameter in Native Transfer Path

Correctness Low Version 1 Code Corrected

CS-ASV-024

`TransferHelperClient._transferToTransferHelper(address from, address asset, uint256 amount)` ignores the `from` parameter when `asset` is native. The native branch transfers amount from the contract's own balance via `TRANSFER_HELPER.call{value: amount}("")`, regardless of the specified `from`. The EVM has no mechanism to pull native ETH from an arbitrary address, so this parameter cannot be honored. The only current call passes `msg.sender`, making this a no-op inconsistency. A future call site passing a different `from` would silently succeed with the transfer originating from the contract itself rather than the specified address.

---

#### Code corrected:

The native branch was removed from the three argument `_transferToTransferHelper` function, native transfers must go through the two argument variant.

## 6.6 Inaccessible Strategy Cannot Be Offboarded or Written Off

Design Low Version 1 Code Corrected

CS-ASV-012

*This issue was independently reported by the client while the audit was ongoing.*

`Allocator._removeStrategy` requires the strategy's share balance to be zero:

```
require(IERC4626(strategy).balanceOf(address(this)) == 0, StrategyStillHasFunds(strategy));
```

If for any reason the balance in a strategy cannot be brought to zero e.g., the strategy pauses withdrawals, locks funds, has a bug, or is upgraded and behaves differently, the strategy cannot be removed. Its balance continues to be included in `_getTotalAssetBalance` and propagates into the system-wide NAV, potentially causing the system to over-estimate total assets.

The `AssetRegistry` implements a distrust mechanism for assets that excludes them from NAV computation. However, this operates at asset granularity: distrusting an asset excludes all strategies for

that asset, not just the inaccessible one. No mechanism exists to selectively exclude a single strategy from NAV.

---

#### Code corrected:

A per strategy `isTrusted` flag and corresponding `distrustStrategy()` function has been added. Distrusting a strategy excludes it from `getTrustedAssetBalances()` (used for NAV computation) without affecting other strategies for the same asset. Removal still requires an empty balance and cannot be performed on a distrusted strategy, since that would require trusting its `balanceOf()` reading.

## 6.7 Missing Reentrancy Guard on Allocator

Design Low Version 1 Code Corrected

CS-ASV-010

`Allocator.withdraw()` first consumes the idle balance, then withdraws from the default strategy, then iterates remaining strategies until the requested amount is fulfilled. During this sequence, the Allocator holds partially withdrawn tokens that have not yet been forwarded to the `TransferHelper`, and `amountRemaining` is only partially decremented. If this sequence reaches untrusted code from any of the external strategies, that code can call back into the system while this intermediate state is observable.

Several paths that reach external strategies carry no reentrancy guard: `rebalance()` on the Allocator itself, and the restricted paths through `StableVault.claimSurplusInterest()` and `EarningChainGateway.pushFundsToAccountingChain()` which both call into `Allocator.withdraw()`.

The Allocator has no reentrancy guard at all, and none of these paths pass through a `nonReentrant` function, so the reentrancy locks on `StableVault` and `EarningChainGateway` are not set either and their functions can be entered.

A callback during any of these operations can re-enter the system with no reentrancy protection anywhere in the call chain, while the original operation is still ongoing.

For example, on the accounting chain, a callback during `rebalance()` could enter through `StableVault.deposit()`.

Both `StableVault` and `EarningChainGateway` use `ReentrancyGuardTransientUpgradeable`, but these guards only protect against re-entry through the same contract. Since all strategy interactions flow through the Allocator, adding `ReentrancyGuardTransientUpgradeable` to the Allocator itself would close the cross-contract re-entry path entirely. Strategies are trusted, but they remain external systems and the guard is a low-cost defense-in-depth measure protecting against unintended re-entry.

---

#### Code corrected:

The Allocator now inherits `ReentrancyGuardTransientUpgradeable` and the `nonReentrant` modifier has been added to every entry point that deposits into / withdraws from an external strategy.

## 6.8 Reward Tokens May Become Stuck in Allocator

Design Low Version 1 Code Corrected



Reward tokens distributed directly to the Allocator by strategies cannot be recovered in the current implementation. The Allocator contract acknowledges that it may hold reward tokens:

```
// Ensure that the asset is registered in the AssetRegistry.
// This helps avoid withdrawal of an asset that is held in the Allocator that is not intended to be withdrawn by
// users. The Allocator may hold assets it received accidentally or from rewards earned from supplying to
// strategies.
```

If a whitelisted ERC-4626 strategy distributes rewards directly to the Allocator address, or requires the depositor to claim rewards, the current implementation cannot handle this. According to the documentation, the initial strategies handle rewards at the strategy level and externally-managed strategies may be added in the future. Such strategies may not handle rewards the same way.

### Code corrected:

Similar to other contracts in the system, the Allocator now inherits RescuableToken, providing a permissioned token rescue function. Calls to rescue registered assets or strategy shares revert.

## 6.9 Sequencer Grace Period May Be Insufficient Due to Arbitrum Timestamp Bounds

**Design** **Low** **Version 1** **Code Corrected**

CS-ASV-011

Stale oracle data may be trusted immediately after a sequencer restart on Arbitrum, because the post downtime grace period in L2ChainlinkOracleAdapter can be fully absorbed by Arbitrum's [documented timestamp drift](#).

When the sequencer comes back online, `_getSequencerUptimeData()` computes how long the sequencer has been up by comparing the L2 `block.timestamp` against the uptime feed's `startedAt` (an L1 timestamp):

```
uint256 private constant GRACE_PERIOD_TIME_SECONDS = 3600;

function _getSequencerUptimeData() internal view returns (SequencerUptime, bool) {
    (, int256 sequencerStatus, uint256 startedAt,,) =
        AggregatorV3Interface(SEQUENCER_UPTIME_FEED).latestRoundData();
    uint256 timeSinceUp = block.timestamp > startedAt ? block.timestamp - startedAt : 0;
    bool uptimeElapsedGracePeriod = timeSinceUp >= GRACE_PERIOD_TIME_SECONDS;
    return (sequencerStatus == 0 ? SequencerUptime.UP : SequencerUptime.DOWN,
        uptimeElapsedGracePeriod);
}
```

Per Arbitrum's documentation, the sequencer may set `block.timestamp` up to **one hour in the future** relative to L1 time. `GRACE_PERIOD_TIME_SECONDS` is exactly 3600 (one hour). When the sequencer restarts after an outage and its clock is at the maximum allowed future offset, the very first L2 block satisfies `timeSinceUp >= 3600`. In this case the grace period elapses instantly with no chance for Chainlink oracle nodes to push fresh data.

This matters because the grace period is the sole effective defense against stale data after a typical restart. The heartbeat based staleness check in `ChainlinkPriceOracleAdapter._getPrice()` and `ChainlinkChainBalanceOracleAdapter._getChainBalance()` uses a 24 hour heartbeat, which only triggers if the outage exceeded ~24 hours. All historical Arbitrum sequencer outages have been far shorter (45 minutes to ~6 hours).

The sequencer does not need to be malicious for this to occur. Even a partial offset (e.g., +30 minutes) halves the effective grace period.

---

**Code corrected:**

The hardcoded grace period has been increased to 7200 seconds (2 hours).

## 6.10 Stale Surplus Snapshot in `claimSurplusInterest`

**Design** **Low** **Version 1** **Code Corrected**

CS-ASV-022

In `StableVault.claimSurplusInterest`, the surplus is computed once before the withdrawal loop. Each iteration calls `FundsHandler.processWithdrawal`, which mutates `Allocator` state by withdrawing from strategies. The solvency invariant (`vaultAssets >= vaultObligations`) is not re-checked against the post-loop `Allocator` state. If strategy balances are correlated, the pre-loop snapshot may not reflect the actual surplus available after withdrawals.

---

**Code corrected:**

The pre-loop surplus snapshot logic was replaced with a post-withdraw solvency check.

## 6.11 Wrong Parameter Order in Event Emission

**Correctness** **Low** **Version 1** **Code Corrected**

CS-ASV-009

In `ChainBalanceOracle.setChainBalanceOracleAdapter()`, the `ChainBalanceAdapterSet` event is emitted with arguments swapped: `currentAdapter` (the previous value) is passed as the second argument and `adapter` (the new value) as the third, but the event signature expects `newAdapter` second and `previousAdapter` third.

---

**Code corrected:**

The parameter order in the event definition has been swapped.

## 6.12 CRITICAL\_DELAY Discrepancy Between Documentation and Code

**Correctness** **Low** **Version 1** **Specification Changed**

CS-ASV-027

The documentation states `CRITICAL_DELAY` is 21 days for `AccessManager` admin operations and proxy upgrades. `RolesLib.sol:20` defines `CRITICAL_DELAY = 14` days.

---

In **Version 2**, RolesLib.sol has been moved to RolesConfig.sol, which sources the value from a per-environment deployment configuration where the critical delay is set to 1209600 seconds (14 days). The documentation has been updated and now consistently states CRITICAL\_DELAY is 14 days.

## 6.13 Code Inconsistencies and Readability

**Informational** **Version 1** **Code Corrected**

CS-ASV-017

The following inconsistencies and readability issues were identified in the codebase:

1. EarningChainGateway.\_bridgeIouTokenFromAccountingChain() is named as if it only handles IOUs arriving from the Accounting Chain. However, IOUs can also be bridged between Earning Chains. The function is called for any inbound BRIDGE\_IOU\_TOKEN message regardless of the source chain, and \_receiveData discards the sourceChainId parameter. The function name does not reflect this broader usage.
2. TransferHelper.getBalance() and \_transfer() compare the asset against address(0) directly instead of using Constants.NATIVE\_CURRENCY, which is used consistently elsewhere in the codebase for native currency checks.
3. EarningChainGateway exposes a getIouTokenManager() getter, but AccountingChainGateway does not.
4. Allocator.\_withdrawFromStrategy() accepts a receiver parameter, but it is always set to address(this) in every callpath.
5. The return value of Allocator.\_redeemAllFromStrategy() is unused at its call site.

---

### Code corrected:

Items 1, 2, 3, and 4 have been corrected: the function was renamed, Constants.NATIVE\_CURRENCY is now used consistently in TransferHelper, getIouTokenManager() was moved to BaseChainGateway, and the unused receiver parameter was removed from \_withdrawFromStrategy(). Item 5 was not changed.

## 6.14 Documentation Inconsistencies

**Informational** **Version 1** **Specification Changed**

CS-ASV-003

### Inaccurate Descriptions

1. StableVault.\_getActiveSubVaultsObligations(): comment states rounding up obligations favors the protocol. Imprecise phrasing: rounds up to avoid understating liabilities.
2. IIouTokenManager.burnTokens(): NatSpec says "burns tokens and transfers them". Burning destroys tokens, it does not transfer them.
3. IEarningChainGateway.pushFundsToAccountingChain(): NatSpec claims amount "must be in RAY" but code uses native decimals.
4. IEarningChainGateway.exchangeIouTokens(): @return says "tokenOutReceiver" but the parameter is receiver.
5. ISwapper.executeSwap(): NatSpec describes fromAmount/fromAsset/toAsset but the actual parameters are amountIn/assetIn/assetOut.

6. `IAssetRegistry.AssetConfig`: NatSpec references `depositsToAllocatorAllowed` but the field is `depositIntoAllocatorAllowed`.
7. `OwnedMulticall` contract: `@title` says `Multicall3` but the contract is `OwnedMulticall`.
8. `OwnedMulticall` contract: NatSpec claims `Multicall2` backwards-compatibility but the `onlyOwner` modifier breaks it.
9. `Constants.MAX_BPS`: comment says "base number" but `MAX_BPS = 10000` represents the maximum.
10. `StableVault.transfer()`: NatSpec states the transfer "mirrors the accounting outcome of withdraw -> transfer assets -> recipient deposit". This is inaccurate. A transfer is a simplified accounting-only operation that bypasses withdrawal fees, oracle checks, solvency gating, and slippage.

### Missing NatSpec

1. `StableVault` constructor: missing `@param priceOracle`
2. `AccountingChainGateway` constructor: missing `@param chainBalanceOracle`
3. `ChainlinkChainBalanceOracleAdapter` constructor: no NatSpec
4. `ChainlinkPriceOracleAdapter` constructor: no NatSpec
5. `OwnedMulticall.tryBlockAndAggregate()`: missing `@param requireSuccess`
6. `IEarningChainGateway` interface: missing `@title`, `@author`, `@notice` that all other interfaces provide.

### Typos & Grammar

1. `StableVault.UserPosition` struct NatSpec: "an user's position" should be "a user's position"
2. `StableVaultStorage.globalOriginalDepositsRay` NatSpec: "new issue IOUs" should be "newly issued IOUs"
3. `IStableVault.NoSurplusInterestToClaim` error NatSpec: "there are no surplus interest" should be "there is no surplus interest"
4. `IStableVault.getSubVaultIdByRate()`: "no sub-vault does not exist" should be "no sub-vault exists"
5. `IStableVault.getVaultObligations()`: "obligations owned" should be "obligations owed" (twice)
6. `IIouTokenManager.getLockedBalance()`: "which have been bridged" should be "which has been bridged"
7. `IIouTokenManager.getLockedBalance()`: "Lock IOU tokens" should be "Locked IOU tokens"
8. `IChainGateway.receiveMessage()`: "receiving of a data" should be "receiving of data"
9. `IChainGateway.receiveMessage()`: "from which...was sent from" contains a duplicate "from"
10. `IChainBalanceOracle.getChainBalance()`: "balance of asset" should be "balance of an asset"
11. `IChainBalanceOracle.getChainBalance()`: "chain id query data for" should be "chain id to query data for"
12. `AccountingChainGateway` contract NatSpec: "messaging one or more" should be "messaging with one or more"
13. `MathLib.rpow()`: NatSpec describes "x to y" but the parameter is `n`

### Malformed NatSpec



1. `IFundsHandler.addEarningChain()` and `IFundsHandler.removeEarningChain()`:  
@param embedded mid-line inside @dev. These will not parse correctly.
2. `ChainBalanceOracle` contract `NatSpec`: @dev merged into @notice on the same line. This will not parse correctly.

### Duplicate Error Definition

`InvalidSignature` error is defined in both `Errors.sol:45-47` and `IWithdrawalPolicy.sol:25-27` with identical selector (`0x8baa579f`) and description.

---

### Specification changed:

Documentation inconsistencies and typos have been resolved.

## 6.15 ERC4626 Strategy Rounding Loss on Dust Withdrawals

Informational Version 1 Code Corrected

CS-ASV-015

ERC4626 strategies use ceiling division when burning shares. The Allocator holds shares in strategies and redeems them to retrieve stablecoins. This rounding works against the system. When `Allocator._withdrawFromStrategy()` calls `strategy.withdraw(1)`, the strategy burns `ceil(1 / sharePrice)` shares. For any strategy where `sharePrice > 1`, this burns 1 full share worth more than the 1 wei withdrawn. The excess value is lost to the protocol.

A user can trigger this by calling `executeWithdrawal()` with the minimum IOU amount that results in a 1-wei withdrawal after fees:

- **0.5% fee:** ~1.005e21 IOU (RAY) for 6-decimal tokens (USDC), ~1.005e9 for 18-decimal (DAI)
- **5.0% fee:** ~1.053e21 IOU (RAY) for 6-decimal tokens (USDC), ~1.053e9 for 18-decimal (DAI)

The loss per withdrawal is extremely small, but real. Since 1 share is always burned to withdraw 1 wei, the loss scales with share price:

| SharePrice | Loss per Withdrawal |
|------------|---------------------|
| 1.01       | 0.01 wei            |
| 1.10       | 0.10 wei            |
| 2.00       | 1.00 wei            |
| 10.00      | 9.00 wei            |

The withdrawal fee does not meaningfully protect against this. The real protection is execution cost: the system loses the rounding difference, while the attacker loses gas costs (the withdrawal fee is negligible in comparison).

Exploiting this as an attack is economically impractical under current conditions. At current Arbitrum gas prices, the cost to execute a dust withdrawal far exceeds the value lost by the system. The two factors that could change this are significantly lower gas costs or strategies with substantially elevated share prices.

Note: This analysis is based on the StableVault system used as designed with stablecoins. If this code is reused in another configuration, the cost/loss ratio may drop significantly, e.g. for high-value, low-decimal assets.

---

**Code corrected:**

`_withdrawFromStrategy()` now uses `redeem(previewWithdraw(amount))` instead of `withdraw(amount)`. The rounding surplus from share ceil division is captured into the Allocator's idle balance instead of being lost to the strategy.

## 6.16 Ineffective Payable Declarations

**Informational** **Version 1** **Code Corrected**

CS-ASV-018

The following functions are declared payable but never receive native currency, making the declaration ineffective:

1. `CcipAdapter.publishMessageToChainWithFeePayer()` is protected with the `onlyGateway` modifier, and the Gateway does not pass native currency along with the call.
2. `AccountingChainGateway.sendPushFundsToChainMessage()` is protected with the `onlyFundsHandler` modifier, and the Funds Handler does not pass native currency along with the call.

---

**Code corrected:**

The unnecessary payable declarations have been removed.

## 6.17 Low-Level Call in `_depositToStrategy`

**Informational** **Version 1** **Code Corrected**

CS-ASV-019

The function `_depositToStrategy` in `Allocator` uses a low-level `strategy.call(abi.encodeCall(IERC4626.deposit, ...))` instead of a high-level `IERC4626(strategy).deposit(...)`. A high-level call would enforce an `extcodesize` check on the target and ABI-decode the return data, reverting if either fails. The low-level call bypasses both checks.

The balance-before/after comparison guards against most impact. However, for deposit amounts at or below `maxSlippageAmount` (a `uint8`, maximum 255 wei), a no-op target passes the slippage check silently. In this case, the preceding `forceApprove` creates a dangling token allowance to the strategy address that is never consumed.

---

**Code corrected:**

The low-level call has been replaced with a high-level `IERC4626(strategy).deposit(...)` wrapped in `try/catch`. The `extcodesize` check and return-data decoding revert on a no-op target, so no dangling allowance can arise.

## 6.18 Manager Operation Edge Cases

Informational

Version 1

Code Corrected

Acknowledged

CS-ASV-013

### Frontrunning setUserRate()

`setUserRate()` takes a `UserRateData[]` array and iterates through it, calling `_setUserRate()` for each entry. If any single user in the batch has withdrawn their position before execution, `_setUserRate()` reverts with `NonExistentPosition()` since `oldSubVaultId == 0`, causing the entire batch to fail.

### Token Donation Affecting Rebalance

When an allocation uses `amount == 0`, the Allocator deposits its entire token balance via `IERC20(allocation.asset).balanceOf(address(this))`. A third party could donate tokens to the Allocator before a rebalance executes, inflating the deposited amount. If the strategy has a deposit cap, this could cause the allocation to revert and thus the entire rebalance operation may revert. The manager can avoid this by specifying explicit allocation amounts instead of relying on `amount == 0`. Note that the attack is impractical as the attacker permanently loses the donated tokens.

#### Code corrected:

1. Fixed: `_setUserRate()` now skips users without a position rather than reverting the batch.
2. Aave Labs acknowledged the grieving vector and provided the following explanation:

Regarding the grieving possible by donating funds to the Allocator to front run a max allocation (due to usage of the ``0`` wildcard), the "rebalancer" will follow the suggestion; in common cases the exact amount of the allocation will be specified as a parameter. Additionally, the team will monitor the supply/deposit caps imposed by strategies.

## 6.19 Missing Getters and Setters for Configuration Mappings

Informational

Version 1

Code Corrected

CS-ASV-016

Several contracts expose setters for configuration mappings but no corresponding getters:

- `ChainBalanceOracle.setChainBalanceOracleAdapter()` sets `oracleAdapterByChainId`
- `PriceOracle.setOracleAdapterForAsset()` sets `oracleAdapterByAsset`
- `BaseBridgeAdapter.setDestinationChainAdapter()` sets `_destinationChainAdapterOf`

Notably, `CcipAdapter`, which extends `BaseBridgeAdapter`, does provide getters for its own mappings (`getChainSelector()`, `getChainId()`).

### Missing Setter Functions in Interfaces

Some interfaces define events for setter operations but do not include the corresponding function signatures:

- `IChainBalanceOracle` declares `ChainBalanceAdapterSet` event but no `setChainBalanceOracleAdapter()` function

- `IPriceOracle` declares `OracleAdapterSet` event but no `setOracleAdapterForAsset()` function

The setters exist in the implementations but are not defined at the interface level.

---

#### Code corrected:

Getters have been added. The missing setter signatures on `IChainBalanceOracle` and `IPriceOracle` are intentional, as the interfaces are scoped to the consumer-facing API.

## 6.20 No Events Emitted on Configuration Updates and Administrative Actions

**Informational** **Version 1** **Code Corrected**

CS-ASV-026

Several setter functions modify critical configuration state without emitting events, making it harder for off-chain systems to track changes:

- `WithdrawalPolicy.setSigner()` updates the fee discount signer allowlist.
  - `BaseBridgeAdapter.setDestinationChainAdapter()` sets the destination chain adapter mapping.
  - `CcipAdapter.setChainSelector()` sets the CCIP chain selector mappings.
  - `IRescuableNative.rescueNative()` and `IRescuableToken.rescueTokens()` perform administrative rescue actions.
- 

#### Code corrected:

Events have been added.

## 6.21 Stale @custom:selector Annotations

**Informational** **Version 1** **Code Corrected**

CS-ASV-008

Four `@custom:selector` annotations carry selectors that no longer match their error signatures. All four stem from error renames where the annotation was not updated to reflect the new name.

- `IFundsHandler.OnlyStableVault()`: annotated `0x93ce7047`, computed `0x53dad68c`
- `IFundsHandler.ChainIdAlreadyPresent()`: annotated `0x148e7b23`, computed `0xff514c10`
- `IFundsHandler.ChainIdNotPresent()`: annotated `0xa80e441d`, computed `0x20be9c4b`
- `IAllocator.DepositsToStrategyDisabled(address)`: annotated `0xd7b75095`, computed `0xbd54a981`
- `OnlyStableVault` was renamed from `OnlyBasedBoostedVault`.
- `ChainIdAlreadyPresent` and `ChainIdNotPresent` were renamed from `ChainIdAlreadyAdded` and `ChainIdNotAdded` respectively.

- `DepositsToStrategyDisabled` had its selector swapped with the since-removed `WithdrawalsFromStrategyDisabled` at creation.
- 

**Code corrected:**

All stale annotations have been updated.

# 7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

## 7.1 Dead Roles on Both Chains

**Informational** **Version 2** **Acknowledged**

CS-ASV-004

All function-based roles are defined and granted uniformly on both accounting and earning chains. Only target bindings are chain-specific. This results in roles like `pushFundsToAccountingChain`, `setUserRate`, and `setChainBalanceOracleAdapter` being fully configured on chains where their corresponding contracts are not deployed. These roles remain unused as no target functions are bound to them.

---

### Acknowledged:

Aave Labs states:

The access control configuration – encompassing profiles, role grants, guardian assignments, and timelock delays – is intentionally defined as a single, system-wide setup rather than per chain type. Introducing chain-specific role granting logic would effectively double the configuration surface area, increasing the likelihood of subtle misconfigurations, specially when editing existing setup.

## 7.2 Excess `msg.value` Reverts Bridging Functions

**Informational** **Version 1** **Acknowledged**

CS-ASV-005

In bridging functions, `_transferBridgeFeeToTransferHelper` forwards the entire `msg.value` to the `TransferHelper` and verifies that `msg.value >= bridgeParams.feeAmount`. The bridge adapter subsequently pulls exactly `bridgeParams.feeAmount` from the `TransferHelper`. When `feeToken` is native currency and `msg.value` exceeds `bridgeParams.feeAmount`, the excess remains in the `TransferHelper`. The `assertingTransferHelperBalanceFor` modifier detects the increased balance and reverts with `TransferHelperBalanceNotConsumed`.

The same pattern exists in the following functions:

- `IouTokenManager.bridgeTokens` (user-facing)
  - `EarningChainGateway.exchangeIouTokens` (user-facing)
  - `EarningChainGateway.pushFundsToAccountingChain` (restricted)
  - `FundsHandler.pushFundsToChain` (restricted)
-

#### Acknowledged:

Aave Labs states:

Acknowledge. For the current CCIP integration, any fee buffer is intended to be provided via ``bridgeParams.feeAmount``, while msg.value is expected to match the native fee allocation exactly, so excess ``msg.value`` is an integration/UX issue rather than a protocol issue; we will document this expectation for integrators.

## 7.3 Heartbeat Staleness Check May Underestimate Real-World Data Age Due to Arbitrum Timestamp Drift

Informational Version 1 Acknowledged

CS-ASV-006

The staleness checks in `ChainlinkPriceOracleAdapter._getPrice()` and `ChainlinkChainBalanceOracleAdapter._getChainBalance()` compare two L2 timestamps:

```
if (updatedAt < block.timestamp && block.timestamp - updatedAt >= HEARTBEAT + HEARTBEAT_BUFFER_SECONDS) {
    isStale = true;
}
```

Both `updatedAt` and `block.timestamp` are Arbitrum L2 timestamps. Per Arbitrum's [documented timestamp bounds](#), the sequencer may set `block.timestamp` up to **24 hours behind** real (L1) time.

If the sequencer runs on-time when an oracle update is recorded, then subsequently drifts to the maximum -24h offset, the L2 delta underestimates real-world staleness by up to 24 hours. Combined with the 24-hour heartbeat, data that is **~48 hours stale** in real time can appear as ~24 hours stale in L2 terms. This is just under the threshold.

This is a theoretical consideration requiring the sequencer to drift to the maximum extreme after an oracle update while Chainlink nodes do not push new data for an extended period. Both the sequencer operator and Chainlink DON are trusted entities.

#### Acknowledged:

Aave Labs states:

Acknowledged. The scenario requires the trusted Arbitrum sequencer to drift to the theoretical maximum of 24 hours behind L1 time while Chainlink simultaneously fails to push updates – both entities are trusted.

## 7.4 Inconsistent Withdrawal Receiver Handling Across Chains

Informational Version 1 Acknowledged

CS-ASV-007



`EarningChainGateway.exchangeIouTokens()` accepts a `receiver` parameter, allowing the caller to direct withdrawn assets to an arbitrary address. In contrast, `StableVault.executeWithdrawal()` on the accounting chain requires `user == msg.sender` and transfers assets exclusively to the caller.

---

### **Acknowledged:**

Aave Labs states:

The inconsistency in behavior is acknowledged; there lacks a concrete reason for the behavior to be consistent.

## 8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

### 8.1 Oracle Balance Transmission Does Not Wait for Source Chain Finality

**Note** **Version 1**

Oracle-based balance transmission does not wait for source chain finality, unlike CCIP messages.

Chainlink oracle operators read `EarningChainStateProvider.getState()` on the earning chain and publish the snapshot to the accounting chain. CCIP messages wait for source chain finality before delivering (approximately 15 minutes for Ethereum and 17 minutes for Arbitrum per CCIP documentation). The balance oracle feed, by contrast, publishes without waiting for finality.

If the earning chain reorgs after the oracle has published a snapshot, the accounting chain temporarily acts on a rolled-back balance:

1. An overstated balance could temporarily unlock interest withdrawals via `requestWithdrawal()` or allow `claimSurplusInterest()` when insufficient surplus actually exists.
2. An understated balance could temporarily block withdrawals unnecessarily. No fund loss occurs.
3. `_validateInboundMessageBlockNumber()` validates that the oracle has captured the earning chain state before funds left. If the oracle snapshot was taken from an unfinalized block that later reorgs, this check can pass incorrectly: obligations are reduced (IOUs burned or funds counted locally) while the oracle-reported earning chain balance does not reflect the corresponding outflow, temporarily overstating system solvency.

These effects are temporary and resolve once the oracle publishes a new snapshot from a finalized block. This is acceptable because the oracle carries balance information with no fund movement. The Chainlink DON trust model that underlies the balance oracle is the same one already relied upon for price oracles.

In the `claimSurplusInterest()` case above, the claimed surplus is sent to the treasury address, which Aave Labs operates as a trusted multisig.

### 8.2 Oracle-Bridge Propagation Asymmetry Can Temporarily Block Interest Withdrawals

**Note** **Version 1**

When a user exchanges IOUs for assets on an earning chain via `EarningChainGateway.exchangeIouTokens()`, two updates propagate to the accounting chain at different speeds. The earning chain balance reduction is reflected in the next Chainlink oracle update (order of seconds). The `BURN_IOU_TOKEN` CCIP message that reduces IOU obligations takes longer to arrive (order of minutes). During this window, `totalAssetsRay` on the accounting chain decreases while IOU obligations remain unchanged, temporarily reducing available surplus in `StableVault.requestWithdrawal()` and `claimSurplusInterest()`.

The two functions use different surplus thresholds and are affected very differently:

`requestWithdrawal` checks surplus against IOUs + `globalOriginalDepositsRay` (principal only). It has a large buffer equal to all accrued user interest. Under normal operation with healthy surplus, this has no effect.

`claimSurplusInterest` checks surplus against IOUs + `activeSubVaultObligations` (principal + accrued interest). This threshold is much higher. Even moderate IOU exchanges may temporarily block treasury claims.

If the system is running close to its obligation threshold, a large IOU exchange could temporarily block interest withdrawals until the burn message is processed. `claimSurplusInterest()` may be blocked even under normal conditions due to its tighter margin. Principal withdrawals remain unaffected.

This is documented behavior. The `_validateInboundMessageBlockNumber()` check in `AccountingChainGateway` ensures the burn message is only accepted after the oracle snapshot reflects the outflow, preventing double counting.

## 8.3 Pooled Positions and Principal-First Accounting Considerations

**Note** **Version 1**

The `originalDepositRay` guarantee is tracked per address and consumed principal first via `_getAmountTakenFromOriginalDeposit()`. If multiple positions were to be aggregated under a single address (e.g., through a wrapper contract that tokenizes vault positions as ERC-20), early withdrawers would consume disproportionate principal protection, leaving later withdrawers with their principal reclassified as unguaranteed interest. While `StableVault`'s transfer interface does not directly support third-party pulls, pooling constructions may still be achievable through intermediary contracts. This is not an issue in the intended use of the system but should be considered a constraint for any integration that intends to pool positions. Governance could use `setUserRate()` to disincentivize such constructions by assigning a less favorable rate to the pooling address.

## 8.4 Stablecoins With Blacklist

**Note** **Version 1**

Some stablecoins such as USDC and USDT feature address blacklists. The pooled custody model combined with multiple transfer and redemption paths allows holders of funds subject to blacklisting (e.g., after a hack) to deposit, move their position to a fresh address, and exit in a different stablecoin.

The system provides several mechanisms to transfer value to an arbitrary address: `StableVault.transfer()`, `StableVault.transferAll()`, standard ERC20 transfers of `IouToken`, `IouTokenManager.bridgeTokens()` with arbitrary `iouTokenRecipient`, and `EarningChainGateway.exchangeIouTokens()` with arbitrary receiver. Additionally, both `StableVault.executeWithdrawal()` and `EarningChainGateway.exchangeIouTokens()` accept an `assetOut` parameter allowing redemption in a different stablecoin than deposited.

An attacker can therefore deposit tainted funds, transfer shares or IOUs to a clean address through any of these paths, and withdraw in a different asset.

A secondary risk exists if a stablecoin issuer blacklists a protocol contract. This scenario is documented in the `Stable Vault` specifications: the system cannot reduce existing exposure, but privileged accounts can unset the default bridge adapter (with zero timelock) to prevent bridging additional funds to the affected chain. Deposits revert automatically if a blacklisted contract is involved in the transfer.